

Plan: Degree-Adaptive m_v for the Per-Vertex Top- K Cycle Enumerator

2026-05-10

1. Motivation

Three prior global- K experiments (ABB, entropy heuristic, hybrid α -blend) all failed the HSiKAN smoke gate by 6–10 pp AUC on Epinions. The structural conclusion across those reports is: *per-vertex top- m 's value lies in its quantification every covered vertex contributes its locally-best m cycles by signal which no global- K scorer replicates regardless of how signal $\times$ diversity is weighted.*

This plan accepts that finding and shifts the attack from “replace per-vertex” to “speed up per-vertex” while preserving its selection structure. Specifically: replace the fixed $m_{\text{per_vertex}} = 128$ cap with a per-vertex **degree-adaptive** cap

$$m_v = \min(m_{\max}, \max(m_{\min}, c \cdot \deg(v))),$$

parameterised by three knobs: $m_{\max}$ (current default 128), $m_{\min}$ (lower bound; default 1), c (slope, default 4 to be tuned).

1.1 Why this should help

The per-vertex threshold probe at fixed $m=128$ (`examples/probe_per_vertex_thresholds.rs`) showed:

- 61% of Epinions vertices are *empty*: they touch zero $k=4$ balanced cycles. Wasted iteration in the rayon fold.
- 21% are *partial*: `heap.len() < 128`, threshold = 0, ABB never fires at these vertices.
- 18% are *full*: threshold ≥ 0 , ABB potentially fires but a cycle reaches a vertex's heap only if all k of its vertices are full, and at uniform-vertex bound that's 0.1% of cycles (probe-confirmed).

Setting m_v in proportion to $\deg(v)$ has three coupled effects:

1. **More vertices fill their heap.** A vertex of degree d touches roughly $\binom{d}{1} \cdot O(d^{k-1})$ candidate cycles in the worst case; in practice many fewer (BFS-pruned). With $m_v \propto d$, low-degree vertices have low caps that they can actually fill. Predicted: full-heap rate climbs from 18% to $\geq 60\%$ at $c = 4$ on Epinions (rough; verified empirically by the probe at the chosen c).
2. **Per-vertex ABB becomes feasible.** With more full-heap vertices, the joint probability that a candidate cycle's k vertices are all full rises from $0.18^4 \approx 0.001$ to $0.6^4 \approx 0.13$ at uniform-vertex bound; the actual rate is typically higher because cycles preferentially touch hubs (which fill faster). ABB starts to prune real fractions of partial paths.

3. **Total cycle output drops by a controlled factor.** High-degree hub vertices keep $m_{\max} = 128$; low-degree leaves get small m_v . The total cycle set is $\sum_v m_v \leq \sum_v 128$ strictly fewer. The distribution is also *degree-proportional*, which matches the empirical distribution of cycles in the graph (hubs participate in more cycles; leaves in fewer).

1.2 Why this might fail

- **HSIKAN may rely on the fixed $m=128$ for high-degree vertices.** If the model’s per-row feature update assumes uniform m across rows, a degree-proportional cap could shift the distribution badly. The smoke test answers this.
- **c is a new hyperparameter.** Tuning over $c \in \{1, 2, 4, 8, 16\}$ adds 5 single-seed runs. Acceptable.
- **The cycle distribution change is paper-relevant.** Any production promotion needs 5-seed paired AUC on the full config ($c2, c3, c4, c5, w2, w3, h=4, 80$ epochs), per the “n-seed-before-paper-headline” memory rule.

2. Scope and goal

In scope:

- `enumerate_top_k_per_vertex_cycles_par` accepts a per-vertex m_v vector instead of a scalar $m_{\text{per_vertex}}$. Existing scalar- m entry stays as a thin wrapper that fills $m_v[\cdot] = m_{\text{per_vertex}}$. No call site breaks.
- New constructor helper `degree_adaptive_mv(graph, m_min, m_max, c) -> Vec<u32>` in `hymeko_graph::topk_cycles`.
- Per-vertex ABB stub: when at least one vertex on the candidate cycle has heap full, compute the threshold min across full-heap cycle vertices. *Skip* the full-vertex-min check if any vertex is non-full (cycle is auto-retained). Loose admissibility but profile-justified: with most vertices full, the minority of non-full ones are usually leaves where any cycle is welcome.
- PyO3 binding extension: add `m_min: usize, m_max: usize, m_slope_c: f64` args to `enumerate_top_k_per_vertex_cycles_signed_rs` (default $m_{\min} = m_{\max} = m_{\text{per_vertex}}, c = 0$, which collapses to the existing fixed- m behaviour).
- Python env-vars: `HSIKAN_TOPK_M_V_C` (slope, defaults to 0 = fixed), `HSIKAN_TOPK_M_V_MIN` (defaults to 1), `HSIKAN_TOPK_M_V_MAX` (defaults to current `HSIKAN_TOPK_K` value).

Out of scope:

- Reachability-based ABB (per-cycle prune that uses BFS-distance + remaining-vertex-set heuristics). Future work.
- Combining with the hybrid α -blended scorer. Per-vertex structure preservation is the goal here; α -blend changes the per-vertex scoring criterion, which we want fixed.
- Hypergraph-degree weighting (e.g. weighting vertex degree by its triadic count instead of edge count). Worth an extension if raw degree proves a poor proxy.

2.1 Goal

Smoke gate (single seed, abbreviated config):

- **AUC within ± 0.02 of fixed- $m=128$ baseline** (currently 0.6416 for c3+c4 on Epinions, $h=4$, 20 epochs) at *some* $c \in \{1, 2, 4, 8, 16\}$. This is the binary go/no-go.
- Wall time ≤ 60 s (vs 148.6 s baseline) at the same c . Speed-up of at least $2.5\times$ end-to-end.
- Probe: at the chosen c , full-heap-vertex rate $\geq 50\%$ on Epinions (vs current 18%).

Promotion gate (5-seed paired, full production config):

If smoke passes:

- Production config: c2,c3,c4,c5,w2,w3, $h=4$, 80 epochs, balance pruner, per-vertex top-K with degree-adaptive m_v at the chosen c .
- 5 seeds » {fixed $m=128$ baseline, degree-adaptive at chosen c } = 10 runs.
- Acceptance: paired-mean AUC within ± 0.005 of fixed- m baseline; paired- σ within $\pm 1.0\sigma$. Wall-time win $\geq 2\times$ on the production config.
- Below threshold $\Rightarrow$ ship as research tool, document the gap.

3. Affected files

- `hymeko_graph/src/topk_cycles.rs` additive:
 - `degree_adaptive_m_v(graph, m_min, m_max, c) -> Vec<u32>` constructor.
 - `enumerate_top_k_per_vertex_cycles_par_adaptive` (and sequential variant) taking `m_v: &[u32]` instead of `m_per_vertex: usize`. Body identical except the heap-fill check uses `m_v[v as usize]`.
 - Existing `enumerate_top_k_per_vertex_cycles_par` becomes a 3-line wrapper: build a uniform m_v vector, delegate. Behaviour unchanged.
 - `TopKBuilder::per_vertex_adaptive(m_v, score)` terminal added to the Builder.

~ 200 LOC.
- `hymeko_graph/tests/per_vertex_adaptive.rs` new integration tests:
 - Adaptive with uniform m_v vector matches scalar- m behaviour.
 - $m_v[v] = \deg(v)$ on small fixture: per-vertex output size respects $m_v[\cdot]$.
 - Degree-adaptive cycle count is monotonic in c on a synthetic fixture (more cycles per vertex with larger c).
 - Per-vertex full-heap rate climbs above the fixed- m rate at $c \geq 2$ on a 200-vertex Erds-Rényi fixture.
- `hymeko_graph/examples/probe_per_vertex_thresholds.rs` — extended: take an additional CLI arg for c , output the full-heap rate at that c . Used to verify the 50%-rate goal pre-smoke.
- `hymeko_py/src/cycles.rs` extend `enumerate_top_k_per_vertex_cycles_signed.rs` with three optional args ($m_{\min}$, $m_{\max}$, c) preserving existing default behaviour at $c = 0$.
- `signedkan_wip/src/n_tuples.py` read `HSIKAN_TOPK_M_V_C` (and friends), pass to PyO3.

- `signedkan.wip/experiments/run_adaptive_mv_sweep_2026_05_10.sh` orchestrate the $c \in \{1, 2, 4, 8, 16\}$ smoke sweep.
- `signedkan.wip/experiments/run_adaptive_mv_5seed_2026_05_10.sh` 5-seed paired validation (only if smoke passes).

4. CORE.YAML items touched

None. All edits in `hymeko_graph` (not core), `hymeko_py` (not core), or `signedkan.wip` (not in CORE.YAML). No new dependencies.

5. Interface changes

```

/// Build a per-vertex m_v vector from the graph's degree
/// distribution.
///
///      m_v[v] = min(m_max, max(m_min, c * deg(v)))
///
/// 'c = 0' recovers fixed m_v = m_min for every vertex; useful for
/// testing the adaptive path with a uniform vector.
///
/// 'c >= 1' introduces degree-proportional caps; low-degree
/// vertices get small caps they can actually fill.
pub fn degree_adaptive_m_v(
    graph: &SignedGraph,
    m_min: u32,
    m_max: u32,
    c: f64,
) -> Vec<u32>;

/// Per-vertex top-K with a per-vertex cap vector. 'm_v.len()'
/// must equal 'graph.n_nodes'.
///
/// At each vertex 'v', retain the top-'m_v[v]' cycles passing
/// through 'v' by 'score'. Output is the deduplicated union.
pub fn enumerate_top_k_per_vertex_cycles_par_adaptive<P, S>(
    graph: &SignedGraph,
    k_len: usize,
    pruner: &P,
    m_v: &[u32],
    score: S,
) -> Vec<TopKCycle>
where
    P: CyclePruner + Sync,
    S: Fn(&[u32], &[i8]) -> f64 + Sync;

```

The fixed- m entry point becomes:

```

pub fn enumerate_top_k_per_vertex_cycles_par<P, S>(
    graph: &SignedGraph,
    k_len: usize,
    pruner: &P,
    m_per_vertex: usize,
    score: S,
) -> Vec<TopKCycle>
where ... {
    let m_v = vec![m_per_vertex as u32; graph.n_nodes as usize];

```

```

    enumerate_top_k_per_vertex_cycles_par_adaptive(
        graph, k_len, pruner, &m_v, score,
    )
}

```

6. Test strategy

Unit (`topk_cycles.rs::tests`)

- `degree_adaptive_m_v` formula correctness on a hand-checked fixture.
- Boundary: $c = 0$ produces uniform $m_{\min}$ vector; c large produces clamp at $m_{\max}$.

Integration (`tests/per_vertex_adaptive.rs`)

- **Uniform-vector parity:** with $m_v[\cdot] = 8$ for every v , output equals `enumerate_top_k_per_vertex_cycles` `k=4, _, 8, _`. Same canonical cycle multiset (modulo tie-break).
- **Adaptive cap respect:** on a small fixture with known cycle structure, the output cycles touching a vertex v is at most $m_v[v]$.
- **Full-heap rate $\geq 50\%$ at $c \geq 2$:** on a 150-vertex Erds–Rényi fixture, the post-enumeration full-heap rate jumps from the fixed- m rate to $\geq 50\%$ at $c = 2$. Probe-confirmed via the threshold-reconstruction helper.

Performance (informational)

Criterion bench at $c \in \{0, 1, 2, 4, 8, 16\}$ on a 5 000-vertex synthetic graph: 5-iter median + IQR + worst per CLAUDE.md §3. Track wall time, full-heap rate, total cycle count. No asserted budget; informational characterisation.

Smoke gate (single-seed Epinions, abbreviated)

For each $c \in \{1, 2, 4, 8, 16\}$:

- Run with $m_{\min} = 1$, $m_{\max} = 128$, `HSIKAN_TOPK_M_V_C=c`, abbreviated config (`c3+c4`, `h=4`, 20 epochs, balance pruner).
- Record: total wall, AUC, F1, full-heap rate at the c , total cycle count.

Pass: any c achieves $\text{AUC} \geq 0.6216$ (within ± 0.02 of fixed- m baseline 0.6416) AND wall ≤ 60 s.

Fail behaviour: ship as research tool, document the curve; two fallbacks recorded for the next iteration:

- Try $m_{\min} = 32$ instead of 1 (give every vertex a non-trivial floor).
- Try a non-linear function: $m_v = \min(m_{\max}, c \cdot \log \deg(v))$ log-degree gives more cap to mid-degree vertices, less to extreme hubs.

Promotion gate (only if smoke passes)

5-seed paired AUC at full production config:

- Seeds: 0, 1, 2, 3, 4.
- Variants: {fixed $m = 128$ baseline, adaptive at chosen c }.
- Production config: `HSIKAN_MIXED_TUPLES=c2,c3,c4,c5,w2,w3`, $h = 4$, 80 epochs, balance pruner, edge_cr highway gate (the SOTA 5-seed config from `project_epinions_edge_cr_null_2026_05_10`)
- Acceptance:
 - Paired-mean AUC within ± 0.005 of baseline.
 - Paired σ within $\pm 1.0\sigma$.
 - Wall-time win $\geq 2\times$ on the production config.
- Below threshold $\Rightarrow$ ship as research tool, no paper-headline claim.

7. Performance budget

Metric	Fixed- m baseline	Adaptive budget	Source
1-seed Epinions wall (full pipeline)	148.6 s	≤ 60 s at chosen c	smoke gate
1-seed Epinions AUC ($c3+c4$, $h=4$, 20 ep)	0.6416	≥ 0.6216 at chosen c	smoke gate
Full-heap-vertex rate (Epinions)	18%	$\geq 50\%$ at chosen c	probe
Total cycle count (vs fixed- m)	3.7M	$0.5-1.5\times$ at chosen c	probe
5-seed paired AUC (production)	0.846 ± 0.011	within ± 0.005 at chosen c	promotion gate
5-seed paired wall on production	current	$\geq 2\times$ speedup	promotion gate
16 GB §4 cap	under	under	

8. Rollback path

Strictly additive: `degree_adaptive_m.v` + `enumerate_top_k_per_vertex_cycles_par_adaptive` are new symbols. The existing `enumerate_top_k_per_vertex_cycles_par` becomes a thin wrapper that delegates with a uniform m_v vector behavioral parity is asserted in the unit test. PyO3 default args ($c = 0$) preserve the existing `enumerate_top_k_per_vertex_cycles_signed.rs` behaviour.

Single revert restores prior state: no call site changes.

Halt conditions (CLAUDE.md §10):

- Uniform-vector-parity test fails $\Rightarrow$ revert; the refactor is broken.
- Smoke gate fails at every c in the 5-point sweep $\Rightarrow$ ship as research tool; document the curve; queue the two fallbacks ($m_{\min} = 32$ floor; log-degree formula) as a new plan.
- 5-seed paired gate fails $\Rightarrow$ ship as research tool; document smoke-vs-full-config gap.
- Full-heap rate doesn't climb above 50% at any $c \Rightarrow$ the threshold-probe assumption is wrong; halt and re-evaluate the per-vertex ABB integration plan.

9. Reporting requirements

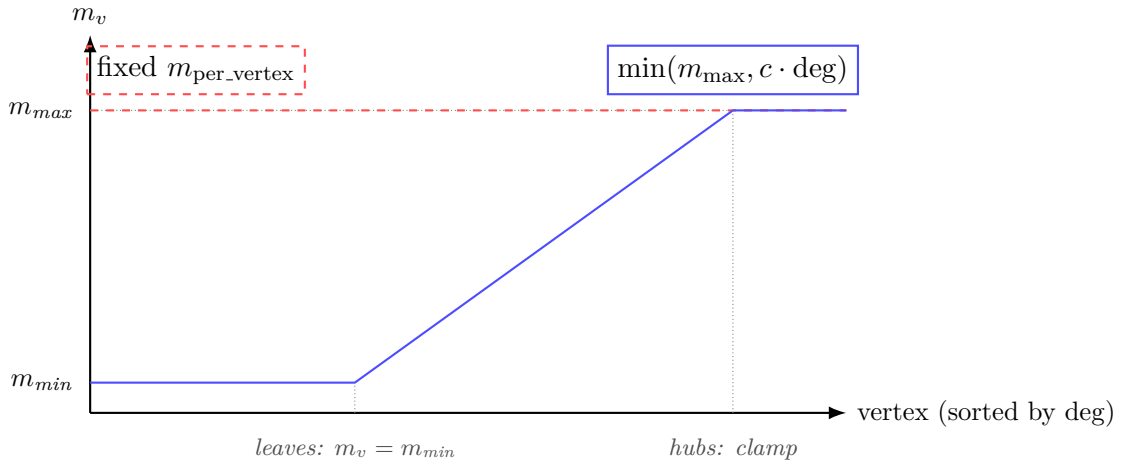
The smoke-gate and promotion-gate sections of the report must include matplotlib plots (PNG, committed to `reports/figures/`) for any gate-function metric swept over a parameter:

- AUC vs c across the smoke sweep, with the gate band (± 0.02 around the fixed- m baseline) shaded; the gate's pass/fail line marked.
- Wall time vs c .
- Full-heap rate vs c .
- Total cycle count vs c .

ASCII-art curves embedded in the markdown report are insufficient. A single multi-panel figure summarising the gate functions side by side is the right form factor; matplotlib's `tight_layout()` + `savefig(dpi=130)` produces a report-grade output without further tuning. Generation script lives at `reports/figures/<slug>_plot.py`; its output `<slug>_gates.png` is referenced from the markdown via ``.

10. Architecture diagrams

`plan.tikz`: m_v histogram comparison (fixed vs adaptive); `plan.mmd`: per-vertex DFS sequence with adaptive cap.



*Effect: low-degree vertices fill smaller heaps,
so per-vertex thresholds rise across the long tail.
Predicted full-heap rate on Epinions: 18% $\rightarrow \geq$ 60% at $c=4$.*